SEARCH IN SNAKE

Laporan Proyek 1

Disusun untuk memenuhi tugas mata kuliah Intelegensi Artifisial Lanjut

Tahun Akademik 2026/2027 Semester 1 (Ganjil)



Disusun Oleh **Kelompok 5**:

Ruhullah Reza Muhammad Khan	23526002
Ananda Myzza Marhelio	23526008
Muhammad Adam Dzulqarnain	23526031

PROGRAM STUDI MAGISTER INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **1**
PEMBAGIAN TUGAS **2**
BAB I
Single Player Evaluation **3**
1.1 Komparasi Algoritma 3
1.2 Analisa Implementasi Algoritma 6
1.2.1 DFS vs BFS 6
1.2.2 BFS vs UCS 7
1.2.3 Greedy Best-First Search vs A* 7
1.2.4 Efek Heuristic pada A* 8
1.2.5 Analisis Kualitas Solusi dan Biaya Komputasi 8
1.2.6 Analisis Keterbatasan dan Kegagalan Algoritma 9
BAB II
Multiplayer Evaluation **10**
2.1 Komparasi Algoritma 10
2.2 Evaluasi Minimax dan Alpha-Beta Pruning 12
2.2.1 Search Depth on Decision Quality 12
2.2.2 Search Depth on Game Performance 12
2.2.3 Search Depth on Execution Time 13
2.2.4 Search Depth on Number of states evaluated 13
2.2.5 Search Depth on Number of states pruned 14
Lampiran **15**
Source Code 15
REFERENSI **16**

PEMBAGIAN TUGAS

NIM	Nama	Kontribusi
23526002	Ruhullah Reza Muhammad Khan	1. Implementasi: a. DFS b. BFS c. Minimax d. Alpha Beta Pruning 2. Mencari nilai metrik untuk Algoritma Multiplayer 3. Memberikan kesimpulan pada 2.2 Evaluasi Minimax dan Alpha-Beta Pruning
23526008	Ananda Myzza Marhelio	1. Implementasi: a. UCS b. Greedy BFS 2. Mencari nilai metrik untuk BFS, UCS, dan Greedy BFS 3. Menyusun dokumen: a. 1.2.1 b. 1.2.2 c. 1.2.5 d. 2.2
23526031	Muhammad Adam Dzulqarnain	1. Implementasi a. A* b. IDA* 2. Mencari nilai metrik untuk DFS dan A* 3. Menyusun dokumen: a. 1.2.3 b. 1.2.4 c. 1.2.5 d. 1.2.6 e. 2.2

BAB I
Single Player Evaluation

1.1 Komparasi Algoritma

Komparasi algoritma dilakukan dengan konfigurasi game sebagai berikut:

1) Board game 20 baris x 20 kolom.
2) Move limit adalah 200.
3) AI speed 4 moves/second.
4) Jumlah apel adalah 10.
5) *Init state* pada setiap algoritma dilakukan secara random tapi dengan *random seed* yang sama setiap percobaannya.
6) Percobaan dilakukan 10 kali.
7) *Success rate* dihitung dari semua apple yang termakan.
8) *Success rate overall* dihitung rata-rata semua percobaan.
9) *Execution time* dihitung dari awal permainan sampai permainan selesai pada akumulasi setiap *choosen_move.*

Algorithm	**Iteration (seed)**	**Number of food items collected**	**Number of states expanded**	**Number of states generated**	**Path length**	**Path cost**	**Execution time (ms)**	**Success Rate**	**Success Rate Overall**
DFS	1 (42)	3	447	882	200	200	3.85	30%	49%
	2 (43)	5	384	788	200	200	3.57	50%	
	3 (44)	4	356	724	200	200	2.71	40%	
	4 (45)	5	775	1506	200	200	6.47	50%	
	5 (46)	5	300	606	200	200	2.03	50%	
	6 (47)	6	302	630	200	200	2.46	60%	
	7 (48)	6	378	770	200	200	2.80	60%	
	8 (49)	5	405	821	200	200	3.68	50%	
	9 (50)	5	379	796	200	200	2.72	50%	
	10 (51)	5	515	1021	200	200	4.79	50%	

Algorithm	Iteration (seed)	Number of food items collected	Number of states expanded	Number of states generated	Path length	Path cost	Execution time (ms)	Success Rate	Success Rate Overall
BFS	1 (42)	10	2052	2718	62	62	17.98	100%	100%
	2 (43)	10	2024	2729	69	69	18.03	100%	
	3 (44)	10	9420	10477	84	84	75.67	100%	
	4 (45)	10	1995	2601	56	56	18.15	100%	
	5 (46)	10	4492	5386	65	65	37.05	100%	
	6 (47)	10	3797	4670	62	62	31.98	100%	
	7 (48)	10	2734	3620	66	66	22.75	100%	
	8 (49)	10	1254	1823	53	53	13.17	100%	
	9 (50)	10	3777	4685	66	66	31.07	100%	
	10 (51)	10	5625	6457	60	60	45.54	100%	

Algorithm	Iteration (seed)	Number of food items collected	Number of states expanded	Number of states generated	Path length	Path cost	Execution time (ms)	Success Rate	Success Rate Overall
UCS	1 (42)	10	6603	9063	64	153	242.08	100%	100%
	2 (43)	10	6381	9172	73	193	238.08	100%	
	3 (44)	10	31203	34743	84	186	1151.87	100%	
	4 (45)	10	8021	10323	60	143	289.90	100%	
	5 (46)	10	15278	18500	63	135	561.79	100%	
	6 (47)	10	8207	10910	60	131	327.05	100%	
	7 (48)	10	6288	8957	66	152	238.79	100%	
	8 (49)	10	2523	4178	53	113	95.24	100%	
	9 (50)	10	10416	13397	66	148	382.79	100%	
	10 (51)	10	18329	21262	60	137	701.79	100%	

Algorithm	Iteration (seed)	Number of food items collected	Number of states expanded	Number of states generated	Path length	Path cost	Execution time (ms)	Success Rate	Success Rate Overall
Greedy BFS	1 (42)	10	289	876	62	62	9.46	100%	100%
	2 (43)	10	310	942	69	69	9.50	100%	
	3 (44)	10	761	2835	84	84	16.65	100%	
	4 (45)	10	247	808	56	56	6.98	100%	
	5 (46)	10	427	1452	63	63	11.66	100%	
	6 (47)	10	307	1043	60	60	9.52	100%	
	7 (48)	10	315	1128	66	66	9.29	100%	
	8 (49)	10	190	645	53	53	6.62	100%	
	9 (50)	10	333	1153	64	64	10.70	100%	
	10 (51)	10	462	1617	60	60	10.04	100%	

Algorithm	Iteration (seed)	Number of food items collected	Number of states expanded	Number of states generated	Path length	Path cost	Execution time (ms)	Success Rate	Success Rate Overall
A*	1 (42)	10	580	1373	65	427	25.43	100%	100%
	2 (43)	10	671	1541	74	142	28.21	100%	
	3 (44)	10	2918	6276	81	138	78.91	100%	
	4 (45)	10	681	1644	61	85	24.38	100%	
	5 (46)	10	838	2030	66	95	29.60	100%	
	6 (47)	10	1018	2428	69	92	34.80	100%	
	7 (48)	10	601	1543	67	77	24.52	100%	
	8 (49)	10	271	739	54	71	13.65	100%	
	9 (50)	10	2434	5371	86	125	72.37	100%	
	10 (51)	10	1171	2811	65	86	36.68	100%	

1.2 Analisa Implementasi Algoritma

1.2.1 DFS vs BFS

Implementasi pada DFS menggunakan *stack* (*Last In First Out*), sehingga sel tetangga yang valid untuk dimasukan ke *stack* adalah sesuai dengan urutan arah (*UP*, *DOWN*, *LEFT*, *RIGHT*). Karena penggunaan *stack*, maka arah yang akan diprioritaskan adalah yang terakhir masuk *stack* yaitu *RIGHT*. Pencarian pada DFS akan mendalami satu cabang sehingga tidak menjamin akan mendapatkan *path* terpendek. Ketika sel di pop, DFS akan menandai sel sebagai *visited*.

Sedangkan untuk BFS, implementasinya menggunakan *queue* (*First In First Out*). Sama seperti DFS, sel tetangga yang valid untuk dimasukan ke *queue* adalah sesuai dengan urutan arah (*UP*, *DOWN*, *LEFT*, *RIGHT*). Namun karena penggunaan *queue*, maka arah yang akan diprioritaskan adalah yang pertama masuk *queue* yaitu *UP*. Sehingga simulasi pencarian *path* mengikuti urutan masuk *queue* dan berjalan per lapisan untuk menemukan path terpendek. Pada BFS, sel akan ditandai *visited* ketika memasuki *queue*.

Pada hasil eksperimen algoritma DFS berdasarkan metrik yang sudah ditentukan, menunjukkan bahwa berdasarkan 10 percobaan *seed* yang berbeda, DFS hanya memperoleh *success rate overall* sebesar 49% dengan rata-rata 4,9 dari 10 apel termakan per *game*. Seluruh percobaan menghabiskan seluruh *turn* yang ada yaitu 200 langkah, dan hanya mampu mengumpulkan sekitar 4-6 apel. Hal ini menunjukan bahwa DFS belum efisien dalam penggunaan *turn* untuk mencapai apel karena penggunaan *stack* tidak menjamin jalur terpendek. Jumlah *state* yang di ekspansi tergolong rendah dengan jumlah pada *range* sekitar 300-800 *states*, dan jumlah *state* yang di *generate* berada pada *range* 600-900 dengan dua iterasi yang memiliki *state* berjumlah lebih dari 1000. Namun DFS memiliki waktu akumulasi *execution time* yang cukup rendah yaitu sekitar 2.03-6.47 ms, sehingga hal ini menunjukan bahwa komputasinya tergolong ringan namun tetap dipengaruhi oleh konfigurasi seed.

Pada konfigurasi yang sama, BFS berhasil mengumpulkan 10 apel pada setiap iterasi sehingga *success rate overall* mencapai 100% dibandingkan dengan DFS yang hanya mendapatkan 49%. Penggunaan *queue* pada BFS membuat ular hanya membutuhkan langkah sekitar 50-90 *turn* untuk memakan seluruh apel, berbanding terbalik dengan DFS yang menghabiskan seluruh *turn* dan tidak menjamin memakan seluruh apel yang ada di *board*. Hal ini menunjukan bahwa BFS lebih efektif dalam mengumpulkan apel dan penggunaan *turn*, berkat *queue* yang dapat mencari *path* terpendek. Meskipun begitu, BFS menghasilkan ekspansi *state* yang lebih banyak dibandingkan dengan DFS yaitu sebesar 1200-9500 *states*. Hal ini juga mempengaruhi akumulasi *execution time* BFS yang lebih besar dibandingkan dengan DFS, yaitu 13,17-75,67 ms, sehingga menunjukan bahwa BFS memiliki komputasi yang lebih mahal daripada DFS namun memastikan bahwa ular dapat memakan seluruh apel yang ada pada *board*.

1.2.2 BFS vs UCS.

UCS umumnya adalah BFS dengan *priority queue*. Implementasi pada UCS menggunakan *priority queue* untuk mencari *path* dari kepala ular menuju apel terdekat, sehingga *state* dengan *cost* terkecil akan diproses terlebih dahulu. Setiap *state* memuat posisi kepala ular dan arahnya, serta *cost* untuk setiap langkah ular yang dihitung berdasarkan *cost turn* ditambah *penalty* berdasarkan jumlah sel tetangga yang kosong (semakin sedikit sel yang kosong, semakin besar *penalty* nya). Penambahan *penalty* bertujuan agar *cost* setiap langkah tidak selalu bernilai satu. Untuk *state* dengan *cost* yang sama, ditangani dengan urutan masuknya *state* ke dalam *priority queue*. *Cost* terbaik atau biaya terendah disimpan pada setiap *state* untuk memastikan bahwa *state* dengan *path* yang lebih mahal akan diabaikan.

Sama seperti BFS, UCS berhasil mengumpulkan seluruh apel pada semua percobaan dengan *success rate* 100%. Namun berdasarkan 10 percobaan yang dilakukan, UCS mengekspansi *state* 195,9% lebih banyak dan menghasilkan *state* 206,6% lebih banyak daripada BFS. Dengan persentase kenaikan ekspansi dan *state* yang di *generate* oleh UCS, *execution time* nya pun bertambah pesat dibandingkan dengan BFS yang mengindikasikan bahwa komputasi UCS lebih mahal dibandingkan dengan BFS. Jika langkah kedua algoritma di rata-rata kan, UCS memiliki rata-rata langkah yang sedikit lebih tinggi dari BFS, yaitu 64.9 langkah dibandingkan 64.3 langkah. Hal ini mungkin terjadi karena penambahan fitur *cost* tambahan yang dimiliki oleh UCS, yaitu penambahan *penalty* yang membuat ruang gerak ular menjadi lebih sempit serta *state* dengan *cost* yang sama ditangani duluan sesuai urutan masuknya ke *priority queue*. Dengan demikian, pada pengujian ini BFS lebih efisien dibandingkan dengan UCS.

1.2.3 Greedy Best-First Search vs A*

Implementasi pada A* menggunakan cost tambahan yaitu penalty, dengan ketentuan (4 tetangga kosong terkena penalty 0, 3 tetangga kosong penalty 1, 2 tetangga kosong terkena penalty 2, 1 tetangga kosong terkena penalty 5, 0 tetangga kosong terkena penalty 6). Cost adalah 1 diambil dari langkah snake ditambah nilai penalty. Greedy BFS pada implementasinya menggunakan nilai heuristic menggunakan metode perhitungan *manhattan distance*. Kedua pendekatan tersebut berpengaruh pada hasil percobaan.

Pada hasil percobaan didapat *number of food collection* dan *success rate* bernilai sama yaitu sepenuhnya 10 makanan dan 100% *success rate.* Greedy BFS memperluas sekitar 67.5% lebih sedikit state daripada A*. Greedy BFS juga menghasilkan *execution time* lebih rendah 72,8%. Hasil tersebut menunjukan bahwa Greedy BFS lebih efisien dari segi komputasi pada percobaan yang telah dilakukan. Hal tersebut konsisten dengan kebutuhan A* yang mempertimbangkan cost yang ditempuh sekaligus nilai heuristic sehingga pencarian memerlukan lebih banyak evaluasi.

Pada percobaan yang telah dilakukan, Greedy BFS menghasilkan jalur lebih pendek dibanding A*. Pada implementasinya penggunaan A* menggunakan penalty cost memilih jalur yang tidak berbahaya bukan hanya jalur terpendek, misalnya jalur yang lebih pendek melewati area beresiko, sedangkan jalur yang lebih panjang melewati area yang dianggap aman. A* dapat memilih jalur yang lebih panjang dengan kondisi tersebut.

1.2.4 Efek Heuristic pada A*

Pada A* pencarian menggunakan cost dan heuristic sebagai pertimbangan. UCS menggunakan cost tanpa nilai heuristic. Pada percobaan ini A* dan UCS menggunakan nilai cost tambahan yaitu nilai penalty berdasarkan tetangga yang kosong. Maka dari itu efek heuristic pada A* dapat dibandingkan dengan data UCS.

Berdasarkan hasil percobaan penggunaan heuristic pada A* dapat meningkatkan efisiensi pencarian dibanding UCS. Rata-rata jumlah *state expanded* menurun 90%. *Excecution time* juga menurun sekitar 91%. Hasil tersebut menunjukan bahwa heuristic pada A* memberikan manfaat dalam efisiensi komputasi.

1.2.5 Analisis Kualitas Solusi dan Biaya Komputasi

Berdasarkan hasil percobaan yang dilakukan, BFS, UCS, Greedy Best-First Search (Greedy BFS), dan A* berhasil mengumpulkan seluruh apel dengan success rate overall 100%, sedangkan DFS hanya mencapai 49%. Jika dibandingkan seluruh algoritma yang memiliki *success rate overall* 100%, Greedy BFS memiliki rata-rata langkah terendah diantara semuanya dengan nilai 63.7 langkah serta rata-rata *execution time* yang paling rendah dengan nilai 10.04 ms menunjukan komputasi yang rendah. Meskipun DFS memiliki *execution time* yang lebih rendah, namun algoritma tersebut membutuhkan 200 langkah tanpa berhasil mengumpulkan seluruh apel di board.

Pergerakan ular pada UCS dan A* mempertimbangkan *penalty* sebagai *cost* tambahan yang mengakibatkan ruang gerak ular yang menyempit sehingga *path* yang dipilih mungkin tidak selalu yang terpendek. Dengan tambahan komputasi tersebut, algoritma belum berhasil memberikan peningkatan *path length* terpendek dibandingkan dengan BFS dan Greedy BFS. Perbandingan *path cost* tidak dapat dilakukan karena *path cost* pada BFS dan GreedyBFS sama dengan *path length* yang tercatat, sedangkan UCS dan A* menambahkan *penalty* pada setiap langkah *cost*-nya.

Hasil percobaan menghasilkan Greedy BFS efisien dalam komputasi dan menghasilkan jalur rata-rata terpendek tetapi tidak menjamin optimalitas. BFS pada percobaan yang telah dilakukan membutuhkan eksplorasi *state* lebih banyak daripada Greedy BFS. UCS memiliki biaya komputasi tertinggi tapi tidak menjamin solusi dengan *cost* minimum pada percobaan ini. A* dengan memanfaatkan nilai heuristic mengurangi eksplorasi yang dilakukan oleh UCS secara komputasi menjadi lebih efektif tapi tidak menjamin *path length* yang lebih pendek. Pada implementasi proyek *snake* ini A* dan UCS menggunakan bantuan cost dengan nilai penalty.

Berdasarkan hasil percobaan Greedy BFS menunjukan kualitas dan biaya komputasi yang paling efektif. A* lebih efisien daripada UCS karena penggunaan nilai heuristic. BFS menghasilkan jalur yang relatif pendek dengan tingkat keberhasilan 100%. DFS belum efektif dalam pencarian solusi karena batasan limit 200 turn. Hasil percobaan menunjukan bahwa biaya komputasi tidak memiliki hubungan linier dengan kualitas solusi yang dihasilkan. Algoritma dengan biaya komputasi lebih tinggi tidak otomatis menghasilkan jalur yang lebih pendek.

1.2.6 Analisis Keterbatasan dan Kegagalan Algoritma

Berdasarkan hasil percobaan keterbatasan setiap algoritma dapat dilihat dari hasil pengujian yang diperoleh. DFS memiliki keterbatasan yang paling jelas pada percobaan ini. Dengan batasan 200 turn, DFS hanya mengumpulkan rata-rata 4-5 makanan dan memperoleh *success rate* 49%. DFS menelusuri satu jalur pencarian secara mendalam sebelum mengeksplorasi jalur lainnya. Strategi tersebut membuat pencarian kurang terarah dan menghasilkan *turn* yang kurang efektif. Kegagagal dalam percobaan DFS dikarenakan adanya keterbatasan 200 *turn*. Oleh karena itu hasil tidak membuktikan bahwa DFS tidak selalu gagal tapi menunjukan keterbatasan pada percobaan yang telah dilakukan.

BFS berhasil mengumpulkan seluruh makanan di semua pecobaan dengan *success rate* 100%. Namun pada percobaan menghasilkan rata-rata 3717 *states expanded* termasuk hasil yang tinggi. Keterbatasan pada BFS adalah banyaknya state yang dieksplorasi sebelum tujuan ditemukan. BFS menghasilkan jalu yang relatif pendek tetapi memiliki biaya komputasi lebih tinggi dibanding Greedy BFS.

UCS mencapai *success rate* 100% tetapi memiliki biaya komputasi yang tertinggi dibanding algoritma lainnya yang diuji. UCS memprioritaskan cost aktual tanpa nilai heuristic hal tersebut menghasilkan UCS dapat mengeksplorasi lebih banyak states. Pada percobaan yang dilakukan cost pada UCS juga dibantu dengan nilai penalty yang juga dapat mempengaruhi pencarian.

Greedy BFS menghasilkan efisiensi komputasi yang baik dibanding dengan algoritma lainnya. Keterbatasan pada Greedy BFS adalah ketergantungan pada nilai heuristic sehingga algoritma ini tidak menjamin dengan biaya minimum.

A* star berhasil mengumpulkan semua makanan dengan *success rate* 100%. Dibandingkan dengan UCS, A* memiliki nilai expanded yang lebih rendah. Pada implementasinya penggunaan A* menggunakan penalty cost memilih jalur yang tidak berbahaya bukan hanya jalur terpendek. Hal tersebut menjadi keterbatasan pada algoritma ini. Karena dapat menghasilkan jalur yang lebih panjang dibanding dengan algoritma lainnya.

BAB II
Multiplayer Evaluation

2.1 Komparasi Algoritma

Evaluasi Minimax dan Alpha-Beta Pruning dilakukan dengan konfigurasi sebagai berikut:

1) Algoritma Minimax/Alpha-Beta Pruning (AI 2) dibandingkan dengan Greedy (AI 1).
2) Masing masing algoritma dijalankan pada *multiplayer game* sebanyak 10 kali.
3) Konfigurasi *board* 20 x 20 dengan 10 *apples* dan 200 *moves*.
4) Minimax dan Alpha-Beta Pruning dijalankan dengan *random seed* yang sama agar hasil dapat dibandingkan secara konsisten.
5) Iterasi dilakukan dengan seed={42, …, 51}.

Minimax								
Search depth	**Iterasi**	**Score**	**Number of states generated**	**Number of states evaluated**	**Number of pruned branches**	**Execution time (s)**	**Status**	**Win Rate**
4	1	5	3388	2178	0	0.051	Draw	20%
	2	4	3318	2180	0	0.078	Lost	
	3	5	2912	1918	0	0.086	Draw	
	4	6	3868	2550	0	0.082	Win	
	5	3	3753	2429	0	0.081	Lost	
	6	5	3947	2557	0	0.087	Draw	
	7	3	4264	2802	0	0.114	Lost	
	8	6	3659	2424	0	0.082	Win	
	9	3	3808	2421	0	0.099	Lost	
	10	2	2636	1747	0	0.054	Lost	
6	1	5	26282	16701	0	0.276	Draw	30%
	2	4	26085	16789	0	0.284	Lost	
	3	5	22854	14781	0	0.225	Draw	
	4	6	29519	18887	0	0.292	Win	
	5	3	26750	16933	0	0.310	Win	
	6	5	29678	19040	0	0.395	Draw	
	7	3	35537	22789	0	0.482	Lost	
	8	6	29456	18892	0	0.416	Win	
	9	3	30837	19456	0	0.362	Lost	
	10	2	21580	13989	0	0.267	Lost	

8	1	5	198923	126536	0	1.523	Draw	20%
	2	4	205108	132022	0	1.644	Lost	
	3	5	180267	116893	0	1.384	Draw	
	4	6	223346	142103	0	1.730	Win	
	5	4	195990	124347	0	1.548	Lost	
	6	5	220976	141339	0	1.627	Draw	
	7	3	268839	172525	0	2.041	Lost	
	8	6	224148	143500	0	1.732	Win	
	9	3	236452	150052	0	1.918	Lost	
	10	2	174766	113123	0	1.334	Lost	

Alpha Beta Pruning								
Search depth	**Iterasi**	**Score**	**Number of states generated**	**Number of states evaluated**	**Number of pruned branches**	**Execution time (s)**	**Status**	**Win Rate**
4	1	5	1920	1048	566	0.053	Draw	20%
	2	4	2133	1238	473	0.069	Lost	
	3	5	1827	1069	407	0.039	Draw	
	4	6	2349	1344	631	0.069	Win	
	5	3	2215	1212	654	0.056	Lost	
	6	5	2293	1268	628	0.066	Draw	
	7	3	2499	1433	643	0.069	Lost	
	8	6	1516	778	688	0.044	Win	
	9	3	2410	1332	535	0.077	Lost	
	10	2	1636	954	407	0.042	Lost	
6	1	5	8214	4139	3062	0.160	Draw	30%
	2	4	9914	5329	2956	0.139	Lost	
	3	5	8791	4756	2712	0.086	Draw	
	4	6	10449	5378	3784	0.175	Win	
	5	3	8355	4062	3443	0.115	Win	
	6	5	10012	5223	3638	0.138	Draw	
	7	3	11558	6093	4090	0.162	Lost	

	8	6	6172	2890	2978	0.126	Win	
	9	3	12089	6299	3391	0.237	Lost	
	10	2	7712	4196	2553	0.107	Lost	
8	1	5	32380	15968	12745	0.420	Draw	20%
	2	4	41584	21459	14115	0.497	Lost	
	3	5	36376	18838	13142	0.395	Draw	
	4	6	40063	19395	15588	0.553	Win	
	5	4	29635	13909	12718	0.432	Lost	
	6	5	38184	18895	16003	0.466	Draw	
	7	3	48718	25352	17142	0.548	Lost	
	8	6	21152	9464	11146	0.207	Win	
	9	3	54606	28125	16204	0.530	Lost	
	10	2	35081	18577	12382	0.360	Lost	

2.2 Evaluasi Minimax dan Alpha-Beta Pruning

2.2.1 *Search Depth on Decision Quality*

Berdasarkan hasil pengujian pada Minimax dan Alpha-Beta Pruning, peningkatan nilai *search depth* tidak secara signifikan menghasilkan kualitas keputusan yang lebih baik. Pada kedua algoritma di *depth* 4, *win rate* mencapai 20% kemudian meningkat menjadi 30% pada *depth* 6 namun turun kembali menjadi 20% pada *depth* 8. Rata-rata *score* juga relatif tetap, yaitu 4,2 pada *depth* 4 dan 6, serta 4,3 pada *depth* 8. Hal ini menunjukan bahwa kemampuan melihat kemungkinan yang lebih dalam belum tentu memberikan peningkatan hasil yang konsisten.

Berdasarkan hasil pengujian pada algoritma Alpha-Beta Pruning, peningkatan *search depth* menunjukkan peningkatan biaya komputasi tanpa diikuti peningkatan performa permainan yang konsisten. Rata-rata *pruned branches* meningkat dari 563 pada depth 4 menjadi 3.261 pada *depth* 6 dan 14.119 pada *depth* 8. Pada depth 4, win rate tercatat sebesar 20%, meningkat menjadi 30% pada *depth* 6, kemudian kembali menjadi 20% pada *depth* 8. Hal ini menunjukkan bahwa efektivitas pruning dalam mengurangi eksplorasi pohon pencarian tidak secara langsung menjamin peningkatan kualitas keputusan, terutama ketika kedalaman pencarian bertambah tanpa peningkatan hasil evaluasi posisi yang sebanding.

2.2.2 *Search Depth on Game Performance*

Peningkatan *search depth* pada Minimax dan Alpha-Beta Pruning belum memberikan peningkatan *game performance* yang baik berdasarkan *score* yang tertera. Pada kedua algoritma di *depth* 4, 6, dan 8, *win rate* masing-masing sebesar 20%, 30%, dan 20%. Sedangkan rata-rata *score*

hanya berubah dari 4,2 menjadi 4,2 dan 4,3. *Depth* 6 memperoleh *win rate* terbaik, tetapi peningkatannya hanya satu kemenangan terhadap *depth* 4 dari 10 percobaan eksperimen sehingga mungkin ruang sampel belum cukup dan relatif kecil untuk menunjukkan keunggulan yang konsisten.

2.2.3 *Search Depth on Execution Time*

Untuk minimax perbedaan execution time berbeda secara signifikan semakin dalam depth sedangkan alpha beta perbedaan depth terhadap execution time tidak sebesar minimax.

Pada Minimax, peningkatan *search depth* menyebabkan kenaikan *execution time* yang sangat signifikan. Rata-rata akumulasi *execution time* pada *depth* 4 adalah 0,0814 detik, kemudian naik menjadi 0,3309 detik pada *depth* 6, dan naik lagi menjadi 1,6481 detik pada *depth* 8. Setiap kenaikan nilai *depth* secara genap meningkatkan waktu sekitar 4x lipat pada *execution time*.

Peningkatan ini terjadi karena setiap tambahan *search depth* memperluas kemungkinan urutan gerakan yang harus ditelusuri dan dievaluasi. Implementasi Minimax tidak memiliki fitur *pruning*, sehingga tidak ada pemangkasan cabang untuk mengurangi pencarian. Hasil tersebut menunjukkan bahwa biaya waktu bertambah secara non linier terhadap *depth*. Sehingga penambahan atau pemilihan *search depth* perlu dipertimbangkan dengan keterbatasan komputasi, terutama dengan peningkatan waktu yang tidak diikuti dengan *win rate* yang konsisten.

Pada Alpha-Beta Pruning, execution time meningkat dari 0,058 detik pada depth 4 menjadi 0,145 detik pada depth 6 dan 0,441 detik pada depth 8. Meskipun waktu eksekusi meningkat seiring bertambahnya ruang pencarian, pruning membantu mengurangi beban komputasi dibandingkan Minimax.

2.2.4 *Search Depth on Number of states evaluated*

Peningkatan *search depth* menyebabkan jumlah *states evaluated* pada Minimax meningkat tajam antar *depth* dengan pertumbuhan secara exponential. Rata-rata *state* yang dievaluasi naik dari 2.320,6 pada *depth* 4 menjadi 17.825,7 pada *depth* 6, kemudian mencapai 136.244 pada *depth* 8. Hal ini terjadi karena pencarian yang lebih dalam menghasilkan semakin banyak kemungkinan urutan gerakan bagi ular, sehingga semakin banyak posisi akhir pencarian yang perlu dinilai oleh fungsi evaluasi. Tanpa fitur *pruning*, Minimax tidak memangkas cabang yang tidak berpotensi. Peningkatan nilai *search depth* tidak menghasilkan peningkatan *win rate* yang konsisten. Sehingga, peningkatan ini hanya meningkatkan beban komputasi saja tanpa menjamin hasil yang lebih baik.

Pada algoritma Alpha-Beta Pruning, rata-rata jumlah states *evaluated* meningkat dari 1.168 pada depth 4 menjadi sekitar 4.837 pada depth 6 dan 18.998 pada *depth* 8. Peningkatan ini menunjukkan bahwa semakin dalam pencarian, semakin banyak *state* yang perlu dievaluasi untuk menentukan langkah terbaik. Meskipun *pruning* mengurangi eksplorasi cabang yang tidak diperlukan, jumlah *state* yang dievaluasi tetap meningkat seiring bertambahnya *search depth*.

2.2.5 Search Depth on *Number of states pruned*

Pada Minimax, algoritma tidak memiliki fitur *pruning*. Pada algoritma Alpha-Beta Pruning, semakin dalam search depth, semakin banyak branches yang berhasil dipangkas (pruned). Rata-rata jumlah pruned branches meningkat dari 563 pada depth 4 menjadi 3.261 pada depth 6 dan 14.119 pada depth 8. Hal ini menunjukkan bahwa semakin luas ruang pencarian, semakin banyak cabang yang dapat dieliminasi sehingga proses pencarian menjadi lebih efisien dibandingkan Minimax tanpa pruning.

Lampiran

Source Code

https://github.com/RuhullahReza/Project-1-Advanced-AI-Search-In-Snake/

REFERENSI

S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.